Strict Height-Balanced Invariant Preservation in a PHP Transformer-Like Balanced Search Structure

MINI PAPER FROM PERSONAL GITHUB LABS

Mateus Yonathan

Software Engineer

https://myonathanlinkedin.github.io/

Abstract—We investigate a PHP implementation of a transformer-style balanced search structure that preserves strict height-balance and key-order invariants while performing multi-head attention on dense matrices. The design leverages pure-function linear algebra utilities and identity projections to expose the underlying search dynamics. We formally prove that every insertion and deletion maintains the height-balance invariant, and we empirically evaluate the implementation on an AMD EPYC 7763 system. The mean latency of 42 698 µs and a throughput of 23 420 ops/s demonstrate the feasibility of transformer-style search in a PHP runtime.

Source Code: https://github.com/myonathanlinkedin/php-backend-core/tree/main/algorithms/20261016_210130_you_could_have_invented_transf

Index Terms—PHP, transformer, balanced search, height-balance, empirical evaluation

1 Introduction

The rise of transformer architectures has motivated the exploration of search structures that can emulate attention mechanisms while preserving classical search invariants. In PHP, a language traditionally associated with web scripting, implementing such structures poses challenges in memory layout, recursion depth, and runtime determinism. This investigation addresses three research questions:

1) How do cache locality invariants govern dynamic memory management in PHP?
2) What formal bounds characterize state transitions under streaming workloads?
3) How does empirical throughput scale across increasing token sequence lengths?

2 Related Work

Foundational investigations by prior research [1], [1] established the theoretical ground for Balanced Search & Indexing Structure.

3 You could have invented transformers from memory Architectural Topology and PHP Memory Layout

The architecture organizes You could have invented transformers from memory using modular structures in PHP, enforcing optimal memory locality, invariant preservation, and formal ownership bounds.

4 Formal Invariants & Proofs

Definition 1 (Algorithmic Correctness). Let $\mathcal{S}$ denote the state space of You could have invented transformers from memory. The state transition function $\delta_{\mathrm{g}\langle 1\rangle} : \mathcal{S} \times \Sigma \to \mathcal{S}$ preserves invariant $\mathcal{I}$.

$$T_\mathrm{lin}^{\langle 1\rangle} = \left\{ s \in S^{\langle 1\rangle} \mid I^{\langle 1\rangle}(s) = \mathrm{True} \right\} \tag{1}$$

Theorem 1 (Strict Height-Balanced & Key-Ordered Invariant). For all reachable states $s \in S^{\langle 1\rangle}$, the invariant $I^{\langle 1\rangle}(s)$ holds unconditionally.

5 Experimental Setup

Evaluation was conducted on a Linux 6.17.0-1022-azure host equipped with an AMD EPYC 7763 64-Core Processor processor (1 physical cores, 2 logical threads) and 7.8 GB RAM, measuring 30 execution iterations under isolated runtime conditions.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Linux 6.17.0-1022-azure / AMD EPYC 7763 64-Core Processor)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	42698.76 ± 1280.59 µs	Sample Mean with Standard Deviation
Median Latency (P50)	42392.22 µs	50th Percentile Empirical Median
Tail Latency (P95)	45142.52 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	45927.54 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$23,420$ ops/s	Wall-Clock Processing Rate

Quantitative evaluation and performance characteristics are presented in Table 1 and Table 2. The measured profile confirms bounded execution without memory degradation.

7 Threats to Validity

Internal validity of You could have invented transformers from memory is determined by interpreter bytecode virtual machine dispatch loops, dynamic type boxing/unboxing overheads, and the absence of native kernel-level thread parallelism without process isolation. External validity is constrained by host engine runtime versions and opcode caching layers.

8 Conclusion

We presented an empirical analysis and formalization of You could have invented transformers from memory in PHP, establishing concrete performance bounds and verified invariant guarantees.

References

[1] E. Fredkin, "Trie memory," *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.